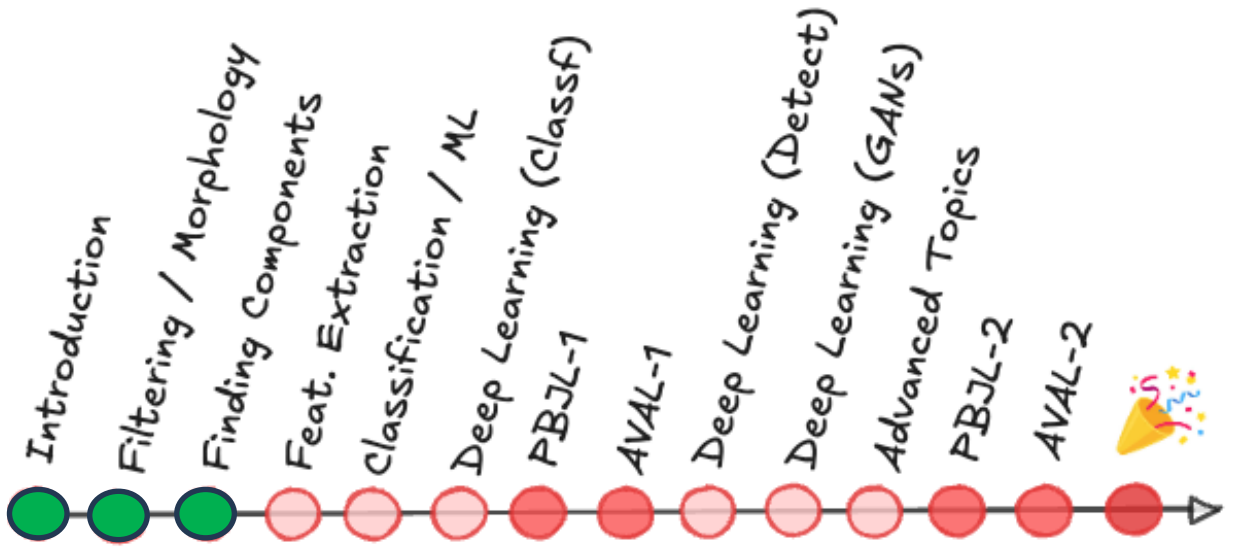
Lecture 04 - Component Segmentation

Prof. André Gustavo Hochuli

gustavo.hochuli@pucpr.br
aghochuli@ppgia.pucpr.br

Topics

- Discussion of Practice 03
- Component Segmentation
  - Finding Connected Components
  - Filtering Components
- Practice
  - License Plate Characters Segmentation



Pipeline Review

1. Input (RGB)

RGB image
(H × W × 3)

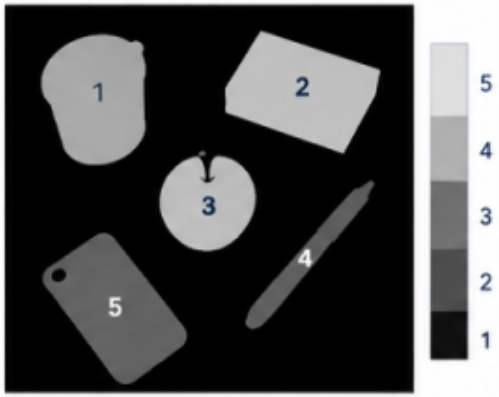
2. Gray

Grayscale image
(H × W)

3. Binary (threshold)

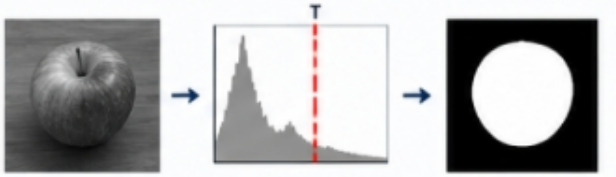
Binary image
(H × W)

4. Connected Components (gray levels)



Connected components
(gray levels 1..N)

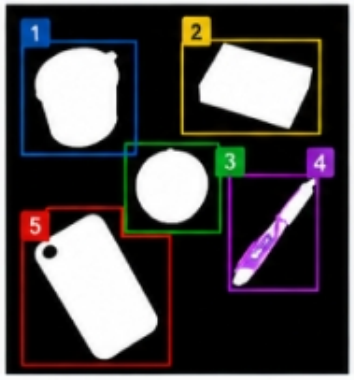
Thresholding



Grayscale (intensity) → Threshold T (e.g., 128) → Binary (foreground = 1)

Pixels with intensity > T become foreground (1), others become background (0).

Connected Components



Connectivity

- 4-connectivity (or 8-connectivity)
- Group adjacent foreground pixels into components.

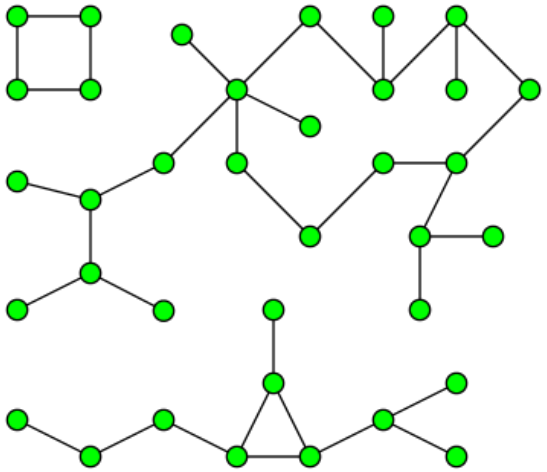
Components (labels / gray levels)

Label	Area (px)	Bounding Box (x, y, w, h)	Centroid (x, y)
1	1248	(28, 28, 60, 58)	(58.1, 48.7)
2	2736	(135, 16, 80, 80)	(175.0, 55.2)
3	420	(55, 100, 30, 15)	(69.5, 107.2)
4	1348	(140, 110, 60, 50)	(168.3, 134.6)
5	984	(25, 160, 45, 45)	(47.6, 182.4)

Component Segmentation

- A.K.A Connected Component Extraction, Blob Extraction, ...

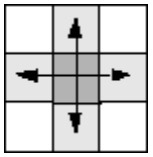
- Its application comes from Graph Theory
  - Social Networks
  - Biology
  - Pattern Recognition



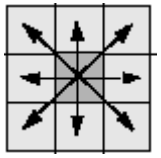
Connected Component Labelling

- Analyzes the non-zero pixel's neighborhood (foreground)
- Label each connected pixel with a label (1,2,3,4....)

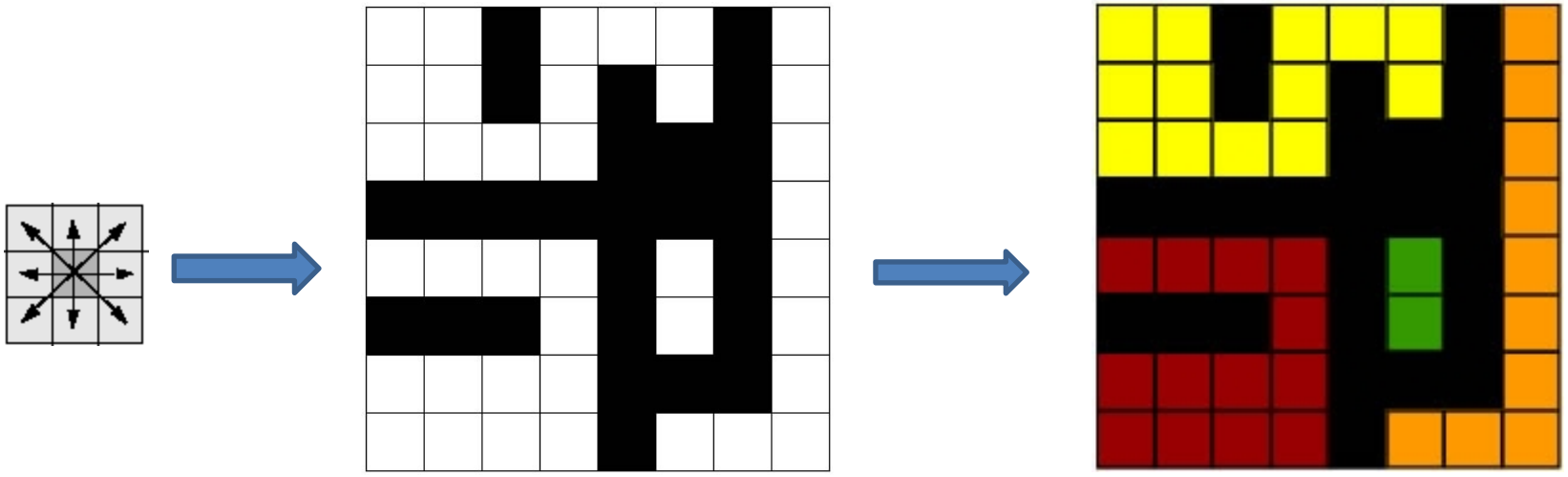
- Kernels:



4-
Neighboors



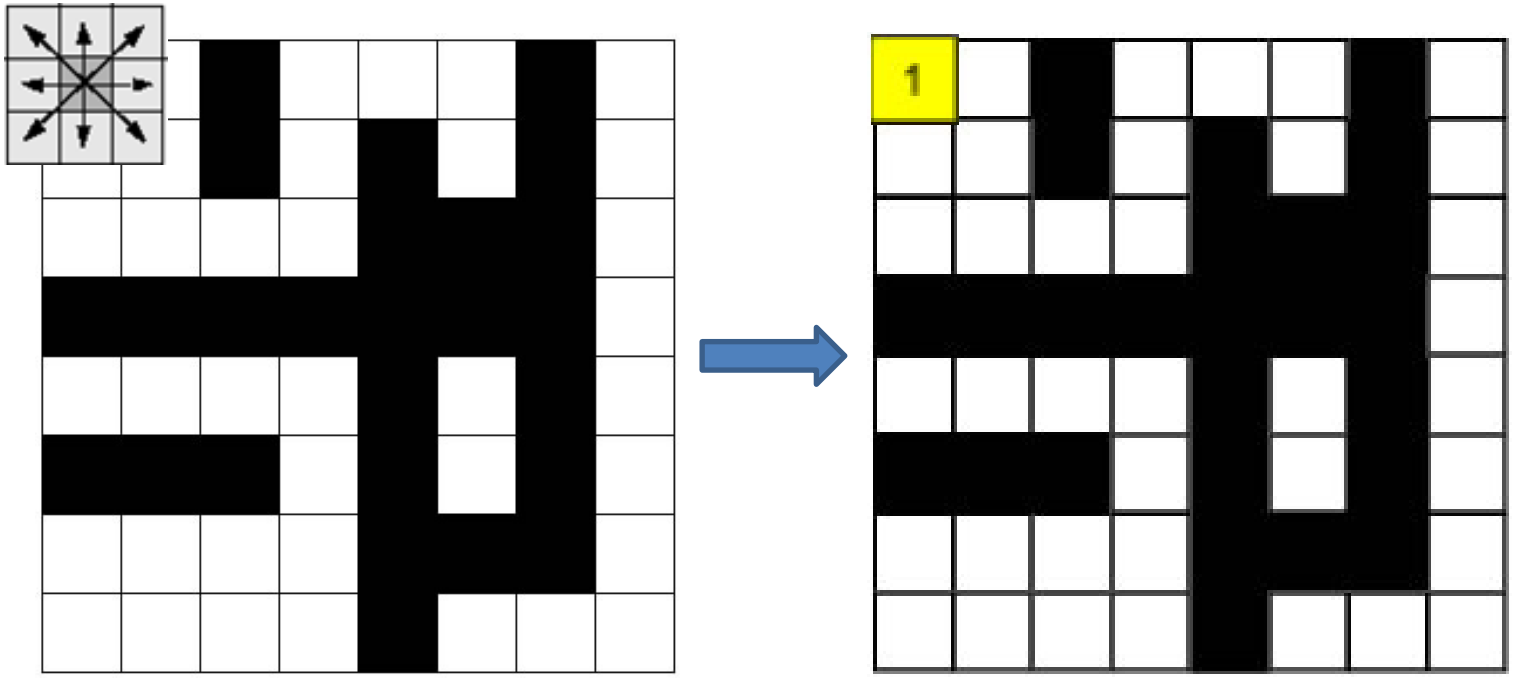
8-
Neighboors



Row by Row Algorithm

- Sliding a connectivity kernel , row by row (2 passes)
  - If the center falls in a non-zero pixel, label it!
  - Labeling:
    - If there are no labeled pixels connected, attribute a new label
    - Otherwise, attribute to it the neighbor´s label.
    - A Union-Find structure control adjacent labels (Union-Find)

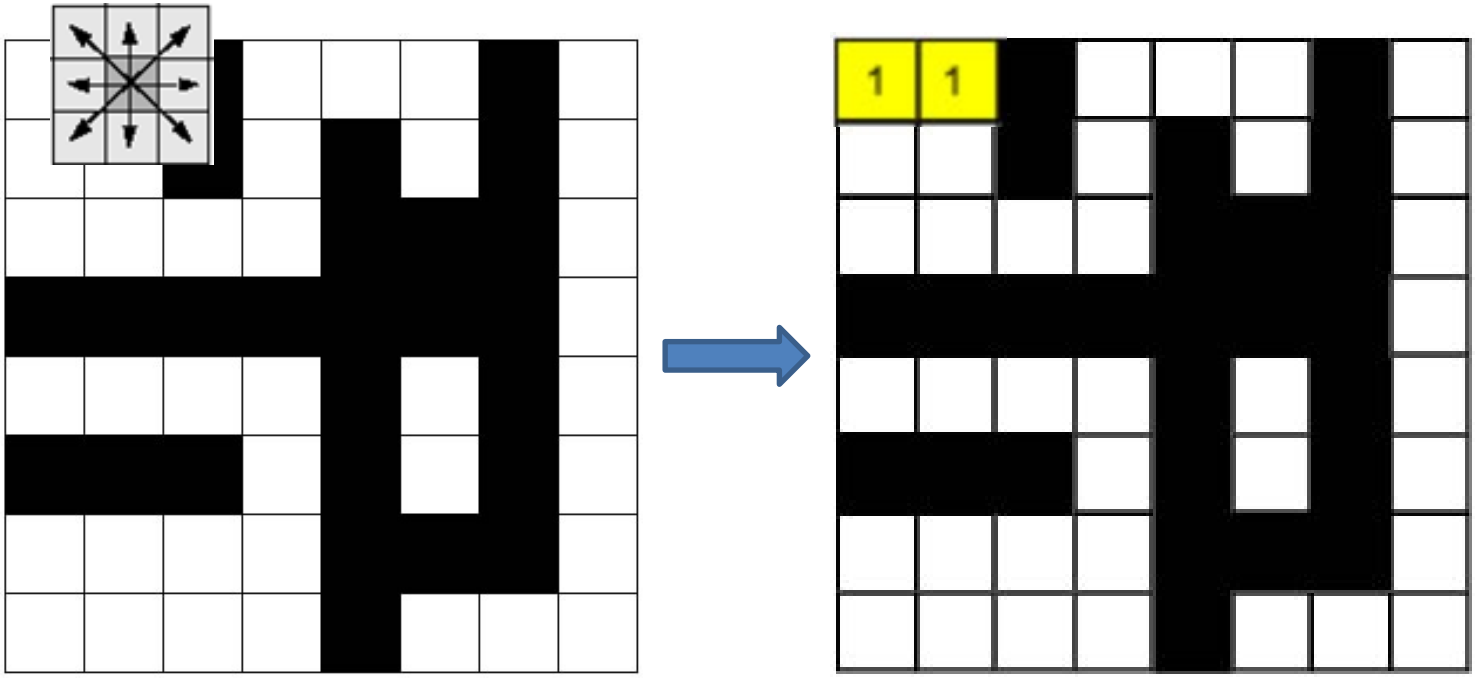
- **Pass #1:**
  - **Row #1**



Row by Row Algorithm

- Sliding a connectivity kernel , row by row (2 passes)
  - If the center falls in a non-zero pixel, label it!
  - Labeling:
    - If there are no labeled pixels connected, attribute a new label
    - Otherwise, attribute to it the neighbor´s label.
    - A Union-Find structure control adjacent labels (Union-Find)

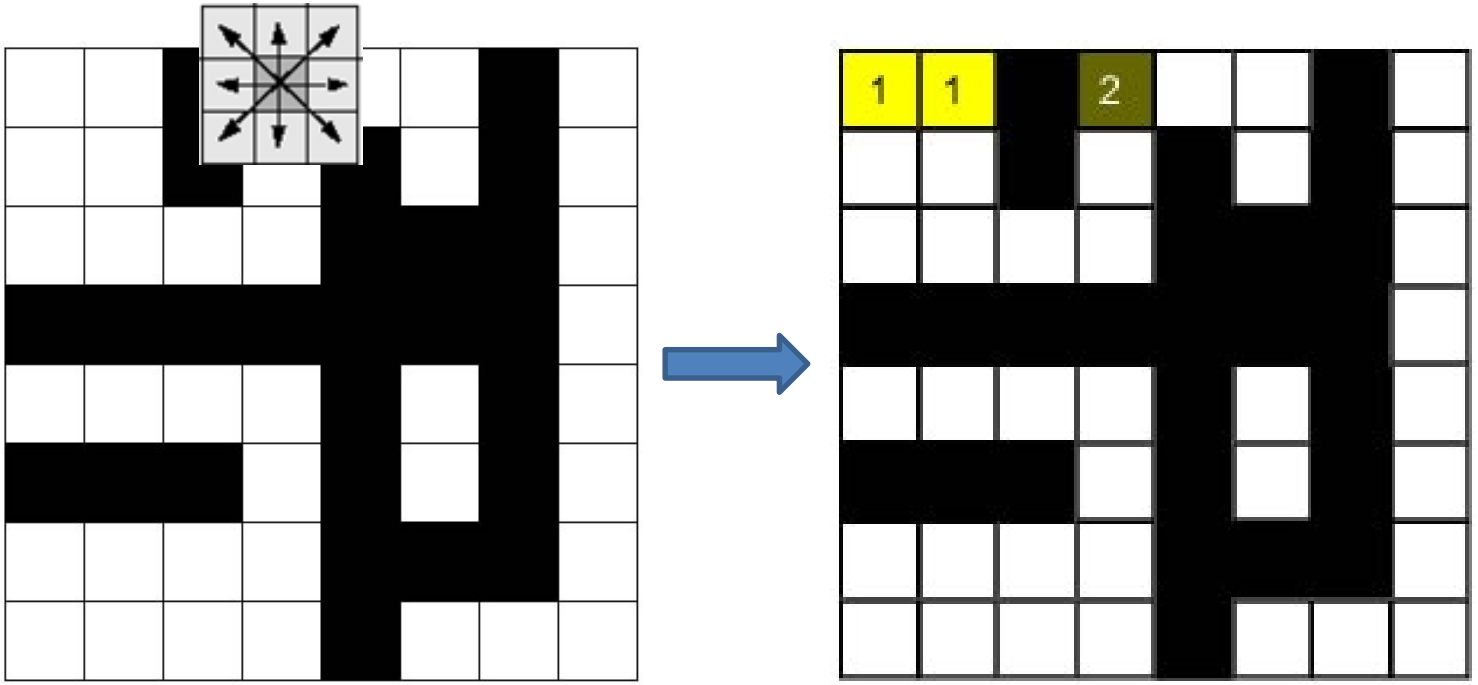
- **Pass #1:**
  - **Row #1**



Row by Row Algorithm

- Sliding a connectivity kernel , row by row (2 passes)
  - If the center falls in a non-zero pixel, label it!
  - Labeling:
    - If there are no labeled pixels connected, attribute a new label
    - Otherwise, attribute to it the neighbor´s label.
    - A Union-Find structure control adjacent labels (Union-Find)

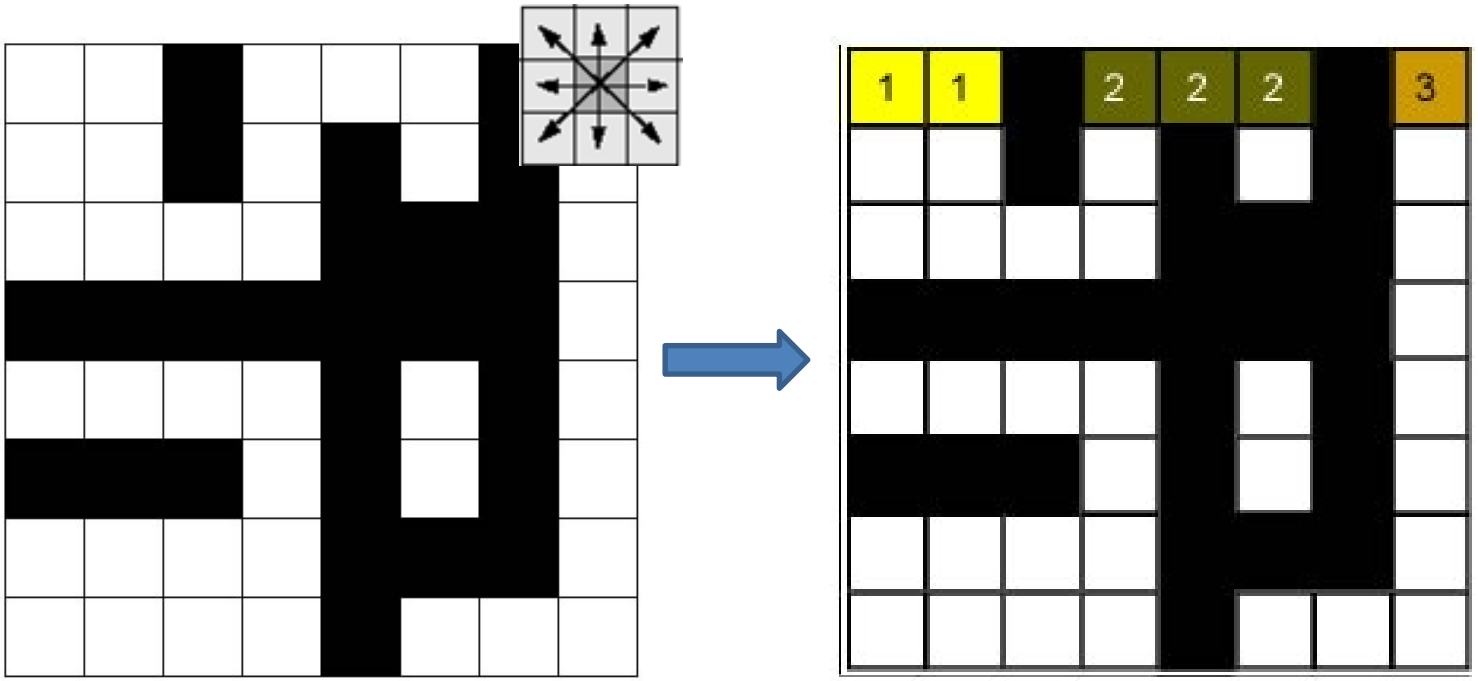
- **Pass #1:**
  - **Row #1**



Row by Row Algorithm

- Sliding a connectivity kernel , row by row (2 passes)
  - If the center falls in a non-zero pixel, label it!
  - Labeling:
    - If there are no labeled pixels connected, attribute a new label
    - Otherwise, attribute to it the neighbor´s label.
    - A Union-Find structure control adjacent labels (Union-Find)

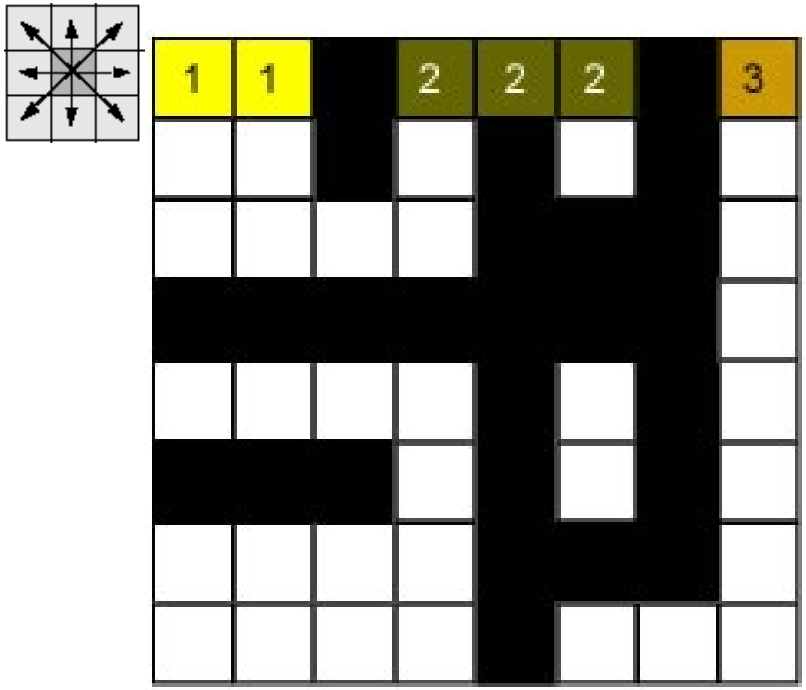
- **Pass #1:**
  - **Row #1**



Row by Row Algorithm

- Sliding a connectivity kernel , row by row (2 passes)
  - If the center falls in a non-zero pixel, label it!
  - Labeling:
    - If there are no labeled pixels connected, attribute a new label
    - Otherwise, attribute to it the neighbor´s label.
    - A Union-Find structure control adjacent labels (Union-Find)

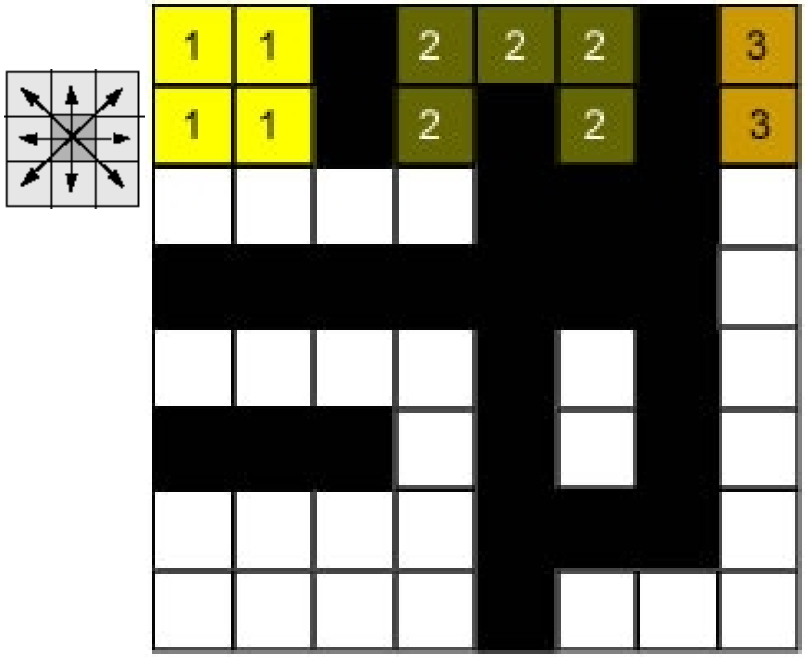
- **Pass #1:**
  - **Row #1**



Row by Row Algorithm

- Sliding a connectivity kernel , row by row (2 passes)
  - If the center falls in a non-zero pixel, label it!
  - Labeling:
    - If there are no labeled pixels connected, attribute a new label
    - Otherwise, attribute to it the neighbor´s label.
    - A Union-Find structure control adjacent labels (Union-Find)

- **Pass #1:**
  - **Row #2**



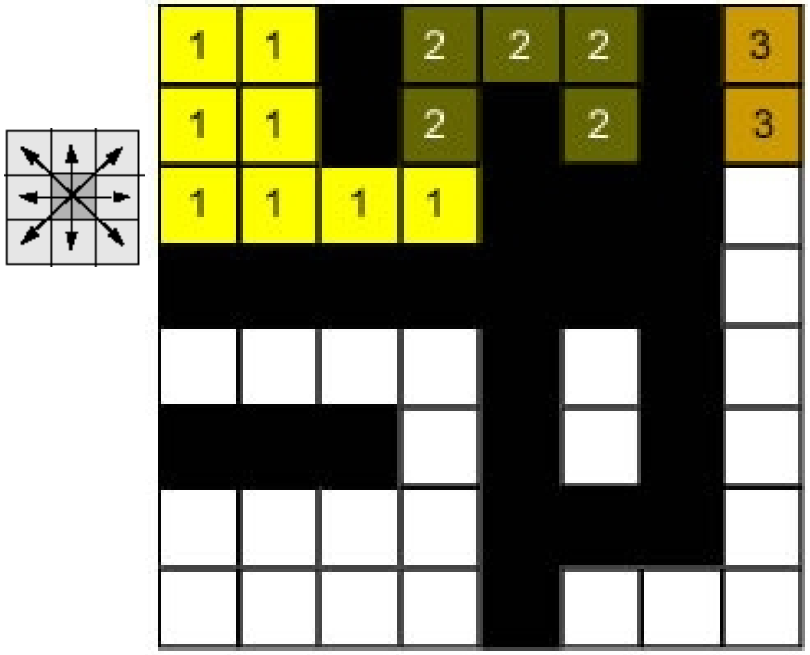
Row by Row Algorithm

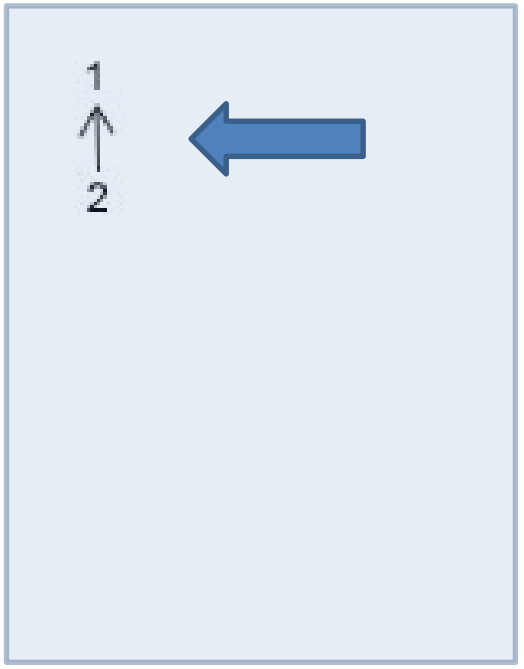
- Sliding a connectivity kernel , row by row (2 passes)
  - If the center falls in a non-zero pixel, label it!
  - Labeling:
    - If there are no labeled pixels connected, attribute a new label
    - Otherwise, attribute to it the neighbor´s label.
    - A Union-Find structure control adjacent labels (Union-Find)

Adjacent labels

Union-Find

- **Pass #1:**
  - **Row #3**

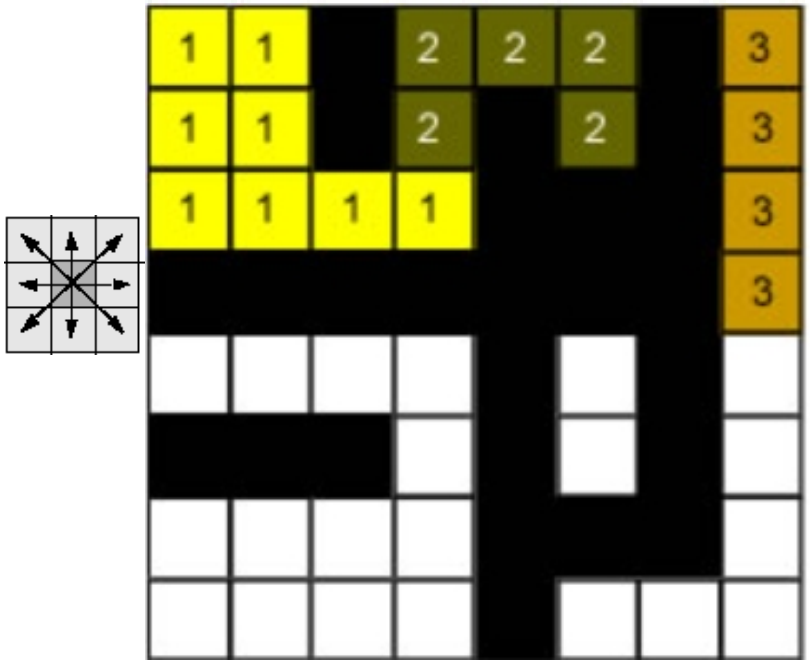




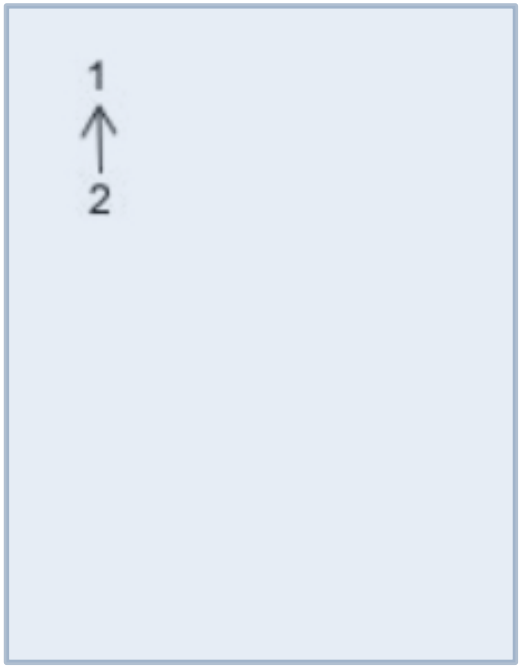
Row by Row Algorithm

- Sliding a connectivity kernel , row by row (2 passes)
  - If the center falls in a non-zero pixel, label it!
  - Labeling:
    - If there are no labeled pixels connected, attribute a new label
    - Otherwise, attribute to it the neighbor´s label.
    - A Union-Find structure control adjacent labels (Union-Find)

- **Pass #1:**
  - **Row #4**



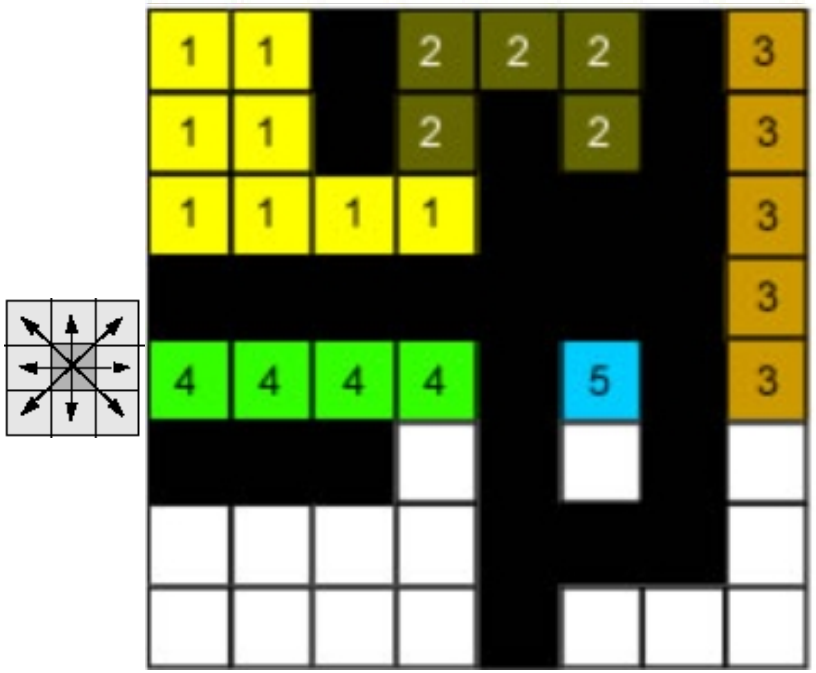
Union-Find



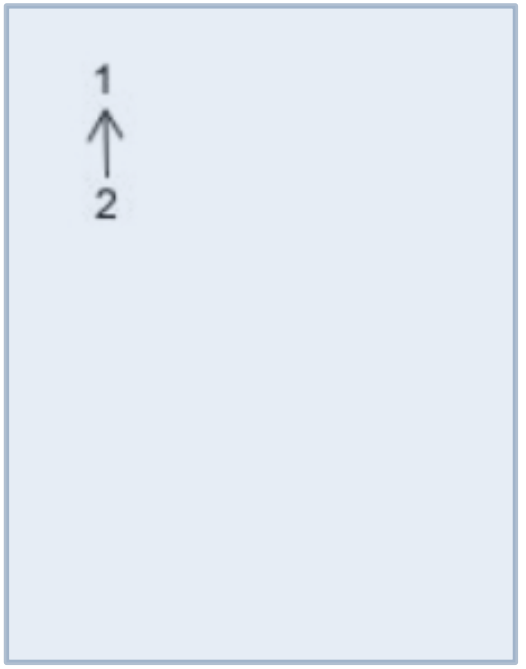
Row by Row Algorithm

- Sliding a connectivity kernel , row by row (2 passes)
  - If the center falls in a non-zero pixel, label it!
  - Labeling:
    - If there are no labeled pixels connected, attribute a new label
    - Otherwise, attribute to it the neighbor´s label.
    - A Union-Find structure control adjacent labels (Union-Find)

- **Pass #1:**



Union-Find

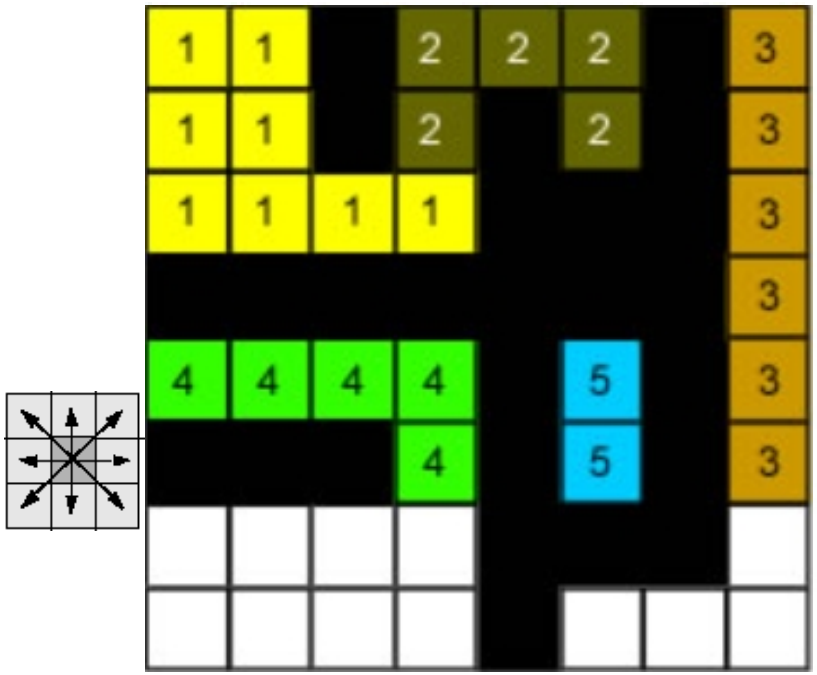


  - **Row #5**

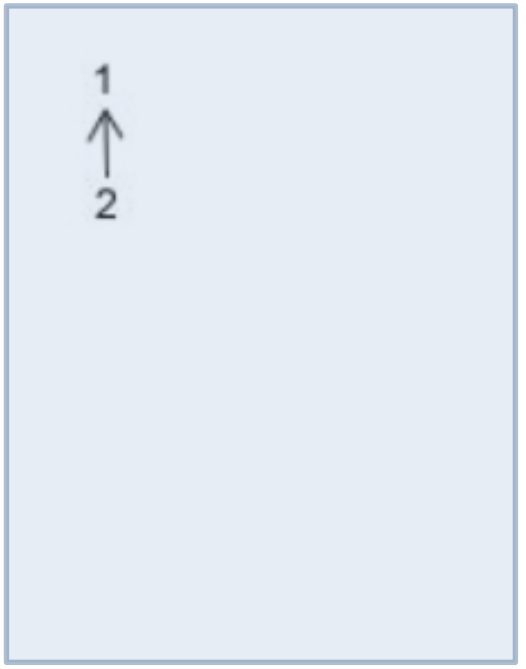
Row by Row Algorithm

- Sliding a connectivity kernel , row by row (2 passes)
  - If the center falls in a non-zero pixel, label it!
  - Labeling:
    - If there are no labeled pixels connected, attribute a new label
    - Otherwise, attribute to it the neighbor´s label.
    - A Union-Find structure control adjacent labels (Union-Find)

- **Pass #1:**



Union-Find

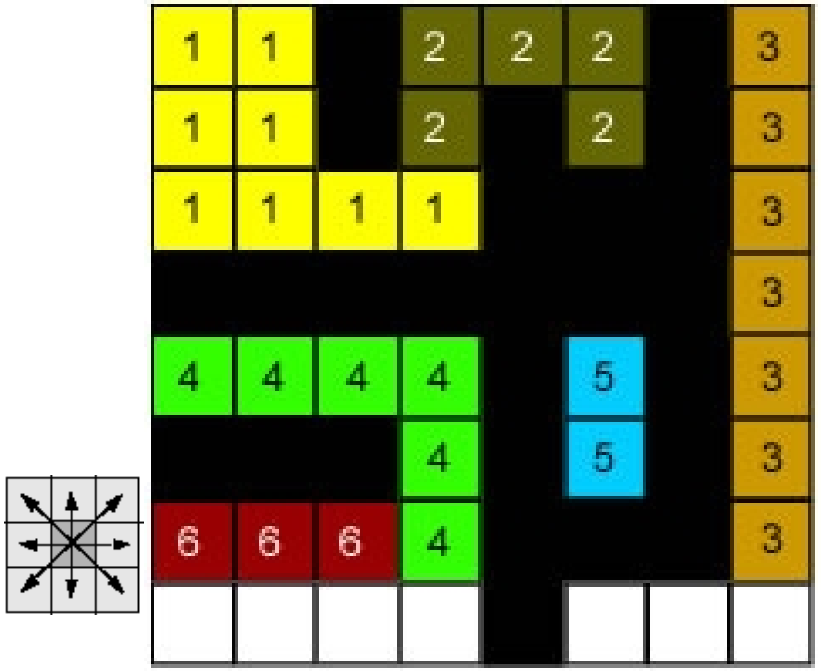


- **Row #6**

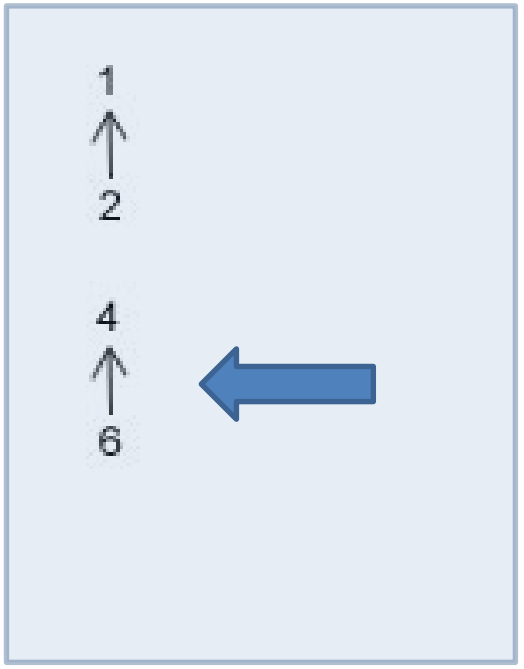
Row by Row Algorithm

- Sliding a connectivity kernel , row by row (2 passes)
  - If the center falls in a non-zero pixel, label it!
  - Labeling:
    - If there are no labeled pixels connected, attribute a new label
    - Otherwise, attribute to it the neighbor´s label.
    - A Union-Find structure control adjacent labels (Union-Find)

- **Pass #1:**



Union-Find

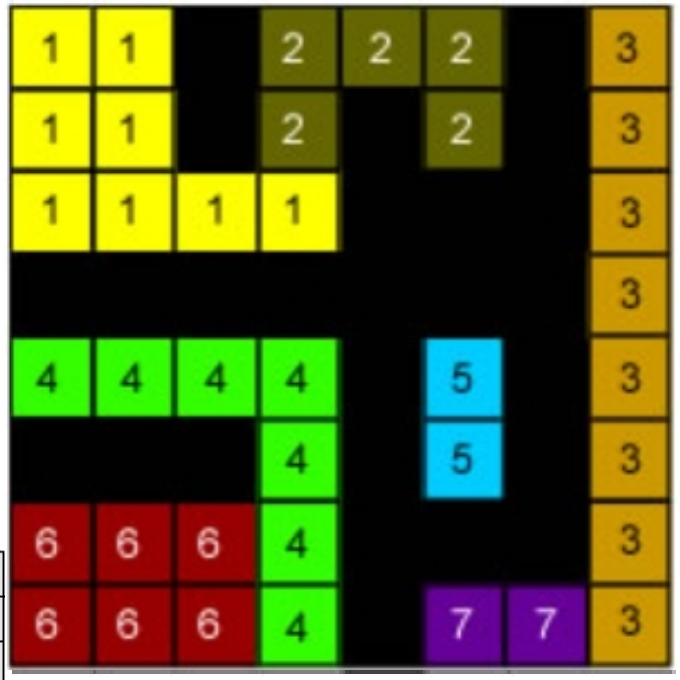


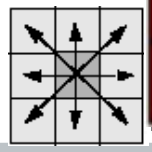
- **Row #7**

Row by Row Algorithm

- Sliding a connectivity kernel , row by row (2 passes)
  - If the center falls in a non-zero pixel, label it!
  - Labeling:
    - If there are no labeled pixels connected, attribute a new label
    - Otherwise, attribute to it the neighbor´s label.
    - A Union-Find structure control adjacent labels (Union-Find)

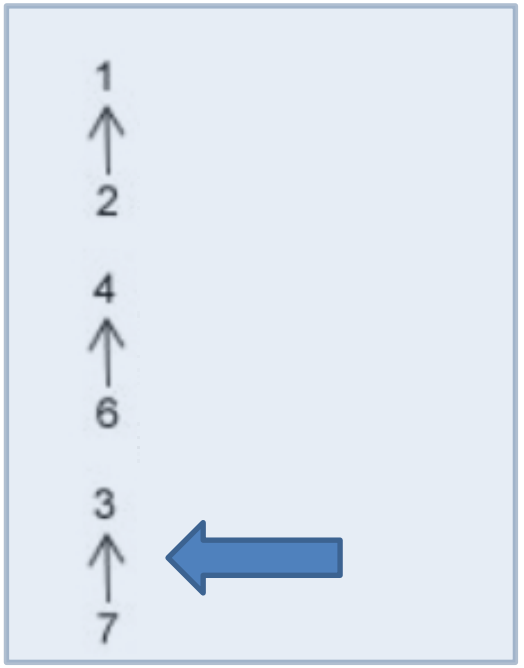
- **Pass #1:**





- **Row #8**

Union-Find

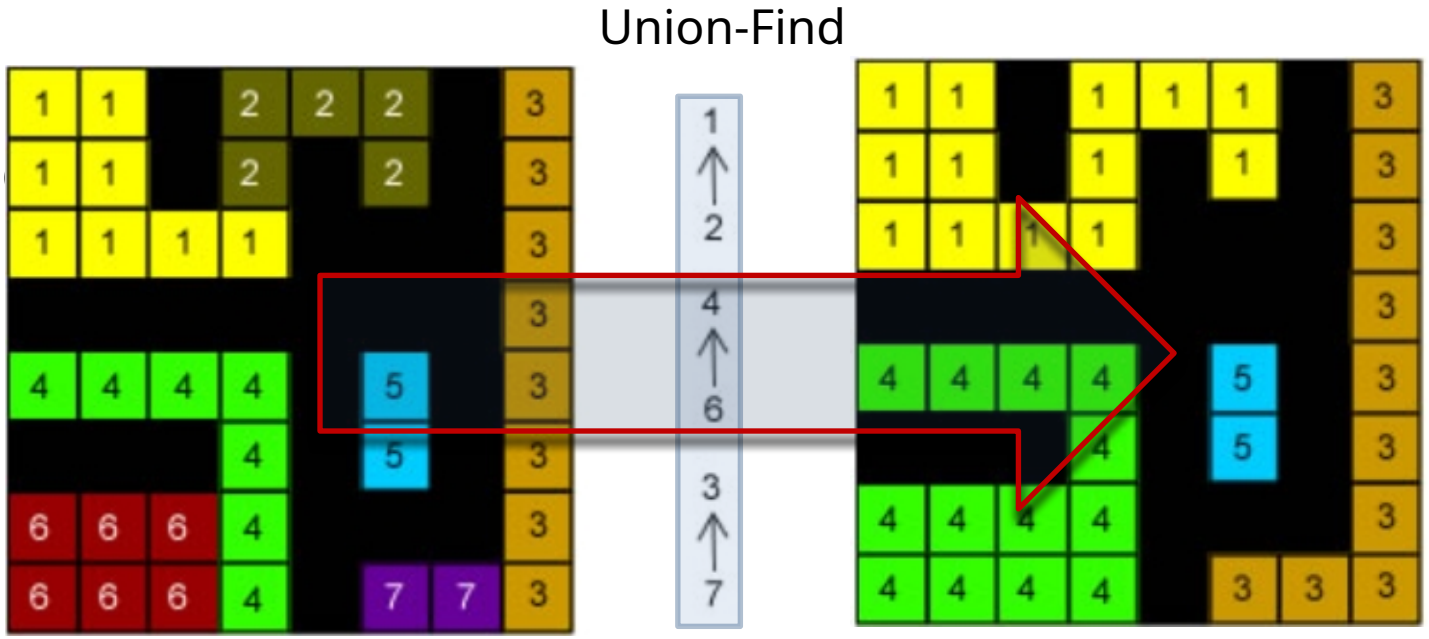


Row by Row Algorithm

- Sliding a connectivity kernel , row by row (2 passes)
  - If the center falls in a non-zero pixel, label it!
  - Labeling:
    - If there are no labeled pixels connected, attribute a new label
    - Otherwise, attribute to it the neighbor´s label.
    - A Union-Find structure control adjacent labels (Union-Find)

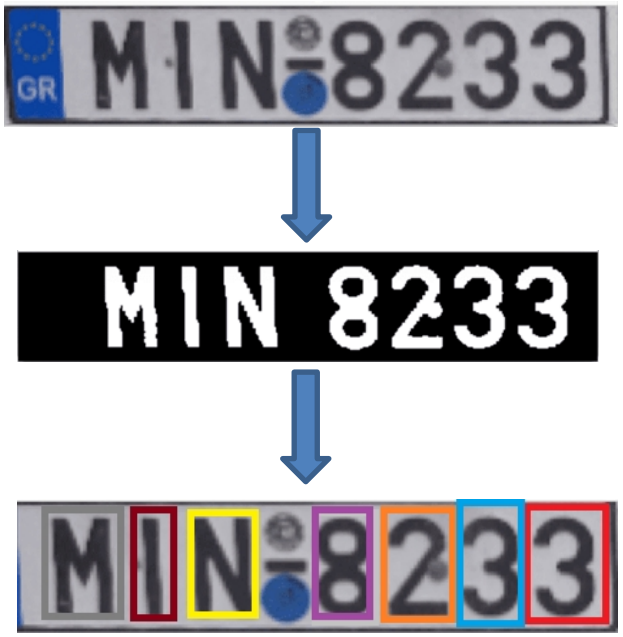
- **Pass #2:**

- **Resolve Union-Fin**



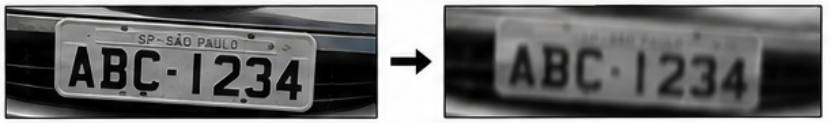
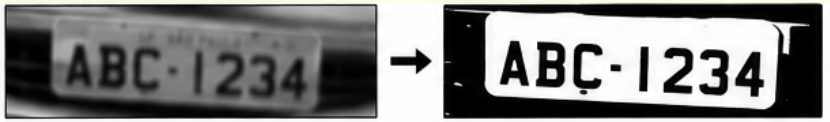
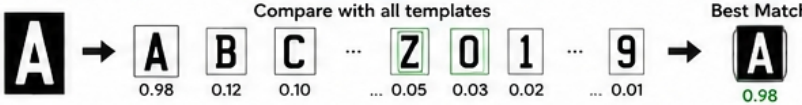
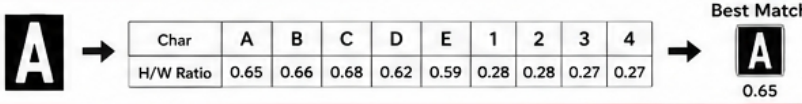
Let’s Code!

- In our pratice, we will implement an algorithm to segment characters in a license plate.



- Besides, we will introduce the cv2.connectedComponent() that implements the component labeling method

- Checkout it here: Lecture 04 - Finding Components.ipynb

PIPELINE STEP	EXAMPLE	WHAT IT SOLVES / WHY IT'S NEEDED
1 Input Image Read image from dataset (.jpg / .png / .tiff)		Solves: • Handles different formats and sizes • Starting point of the pipeline • Real world variations (lighting, angle, noise)
2 Preprocessing Grayscale + Noise Reduction (Gaussian Blur)		Solves: • Reduces color/lighting variations • Removes high-frequency noise • Improves thresholding robustness
3 Binarization (Threshold) Otsu Threshold (Inverted)		Solves: • Separates characters from background • Makes characters uniform (black/white) • Essential for connected components
4 Connected Components Label all connected regions		Solves: • Finds all candidate regions (characters + noise) • Provides stats: x, y, width, height, area • Basis for filtering possible characters
5 Filter by Geometric Features (Height, Width, Aspect Ratio, Area) Keep only likely characters		Solves: • Removes screws, borders, reflections, etc. • Uses features: height, width, aspect ratio, area • Keeps only regions that look like characters
6 Sort Characters (Left to Right) Order by x-coordinate		Solves: • Ensures correct reading order • Important for reconstructing the plate string
7 Normalize Characters Resize to 64×64 with padding (Keep aspect ratio)		Solves: • Makes all characters same size • Keeps aspect ratio and centers character • Required for template matching
8A Template Matching Compare with reference dataset (All A–Z, 0–9) cv2.matchTemplate		Solves: • Recognizes character shape • Uses pixel-wise similarity (structure) • Works well for most printed fonts
8B H/W (Aspect Ratio) Matching Compare height/width ratio (Geometric feature only)		Solves: • Simple and fast • Uses only geometric feature (H/W) • Less accurate (similar ratios for different chars)
9 Reconstruct Plate Combine predicted characters (Left to Right)	From Template Matching: ABC-1234 From H/W Features: ABC-1234	Solves: • Builds final plate string • Allows comparison of methods • Template matching usually more accurate
10 Output / Visualization Show results and bounding boxes	 Template Matching: ABC-1234 H/W Features: ABC-1234	Solves: • Visual confirmation of detection • Helps debug errors • Final output for the user